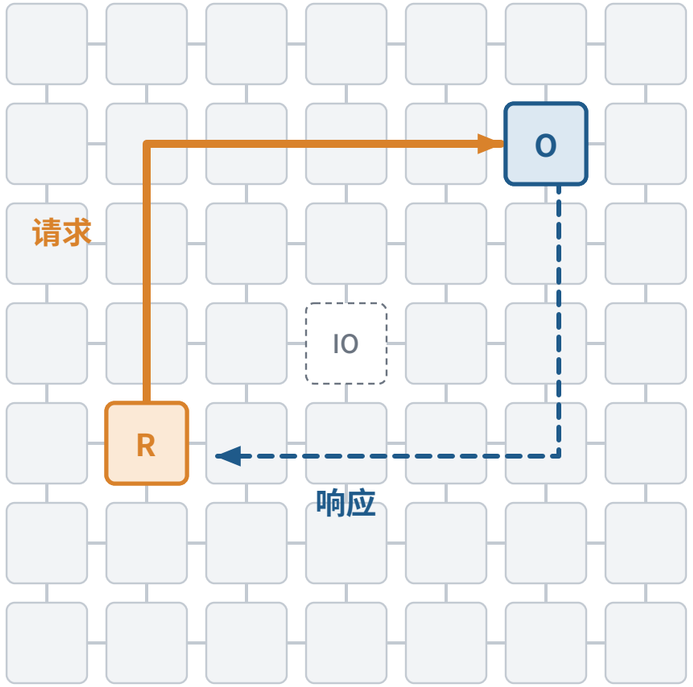
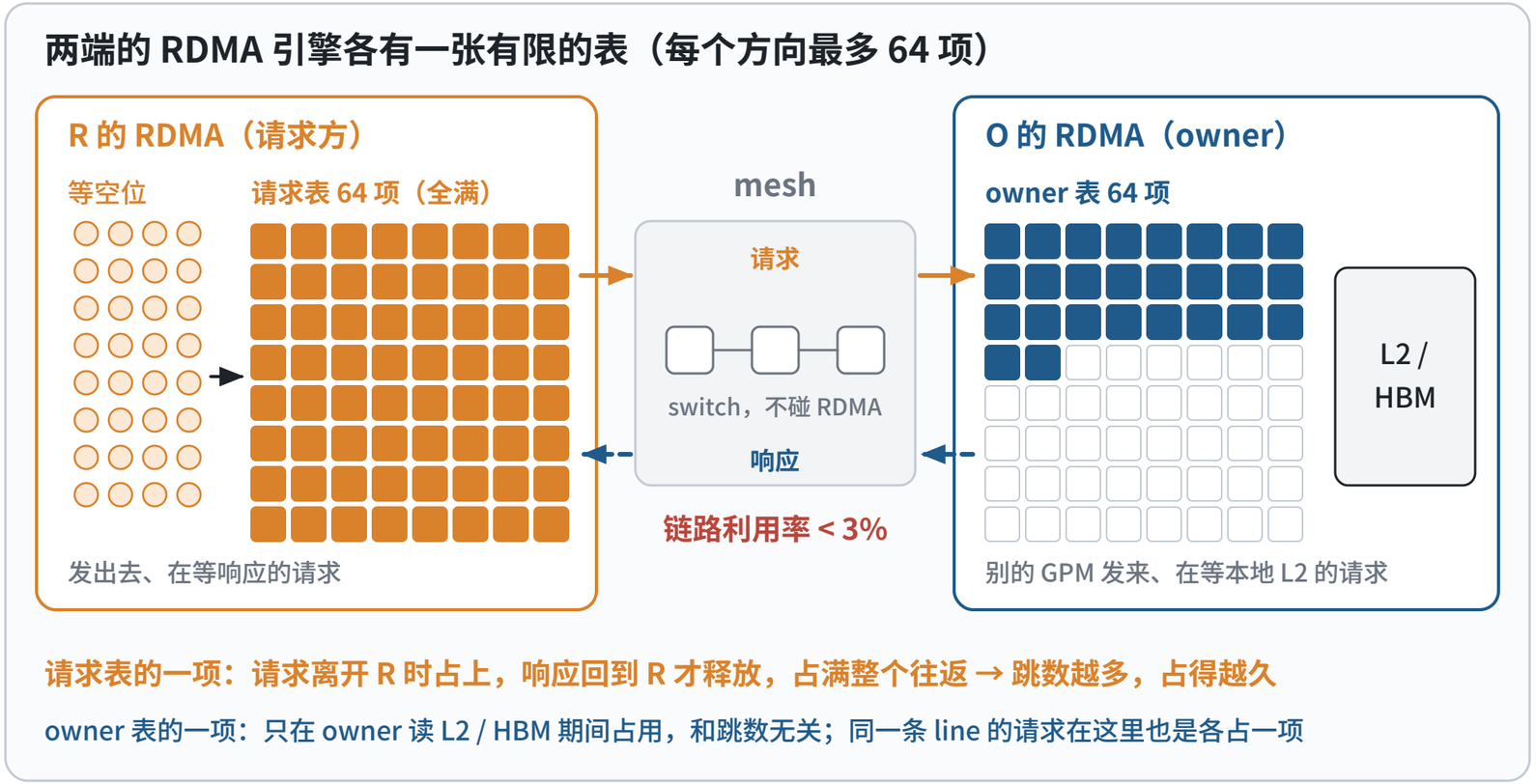
图 1 基线的瓶颈：请求方的 RDMA 引擎只能同时挂 64 个远程请求

RDMA 引擎是每个 GPM 连到 mesh 的网络接口，两端各有一张有限的表；中间每一跳只过 switch，不碰 RDMA



这一次 7 跳：只有 R、O 两端经过 RDMA，
中间每跳只过 switch（约 72 ns / 跳）



带宽被表项卡住

跳数越多，往返越长，每项占得越久：

远程带宽 = 64 × 64 B ÷ 往返时间

1 跳 26.4 B/周期，6 跳只剩 8.0

时间花在排队上

按闭环模型，排队占远程延迟 80–86%

（T17 的三个 workload）

排队就发生在请求表门口

链路是闲的

链路利用率 < 3%

受限的是两端 RDMA 的表，

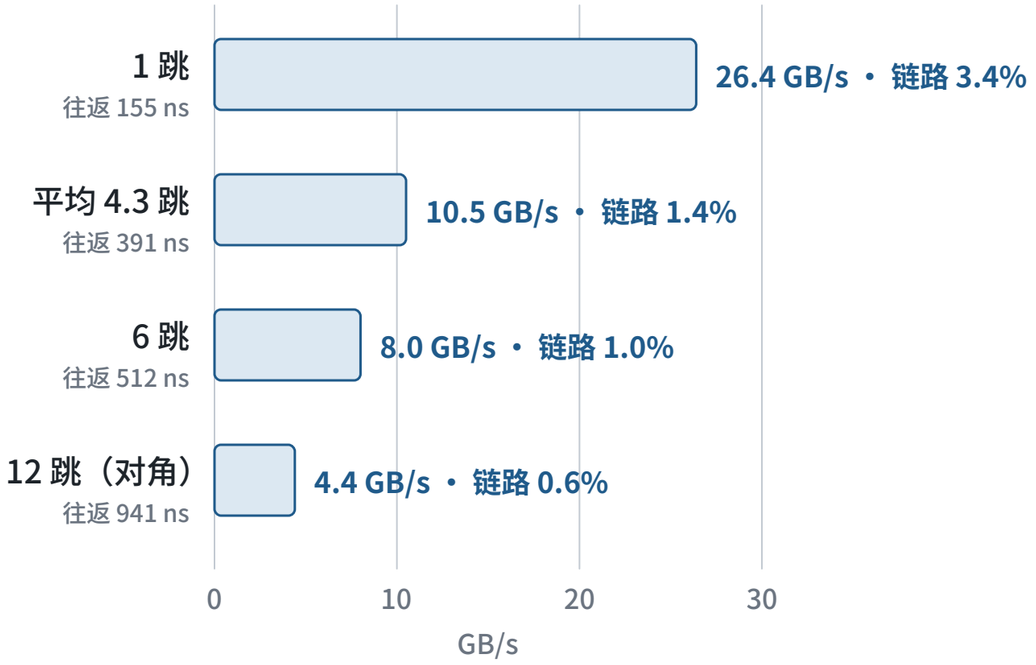
不是链路和 switch

图 2 为什么 wafer 上更严重：64 项在远处撑不起带宽，远的请求还会占满表项

一项从请求发出占到响应回来；跳数越多，每项占得越久

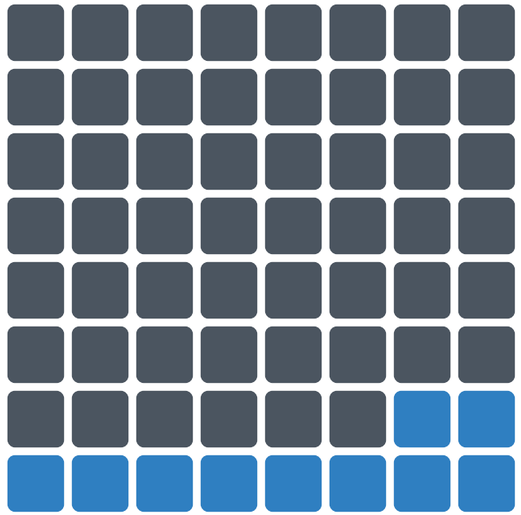
(a) 64 项能撑起的远程带宽

带宽 = 64 × 64 B ÷ 往返时间，往返 ≈ 84 + 71.4 ns × 跳数



(b) 共享 64 项时，远的请求占满表项

例子：一半请求去 1 跳外（往返 155 ns），一半去 10 跳外（往返 798 ns）



远的：54 项

近的：只剩 10 项

表项按"请求速率 × 往返时间"分，
远的每项占得久，就占了 84%；
近的 155 ns 就能完成，
却要排在后面等

2×2 的 MCM 最多 2 跳，往返最多差 1.5 倍；
wafer 上差 6 倍（155–941 ns）

64 这个数不改（当作硬件约束）。三个机制都在 64 项以内想办法：

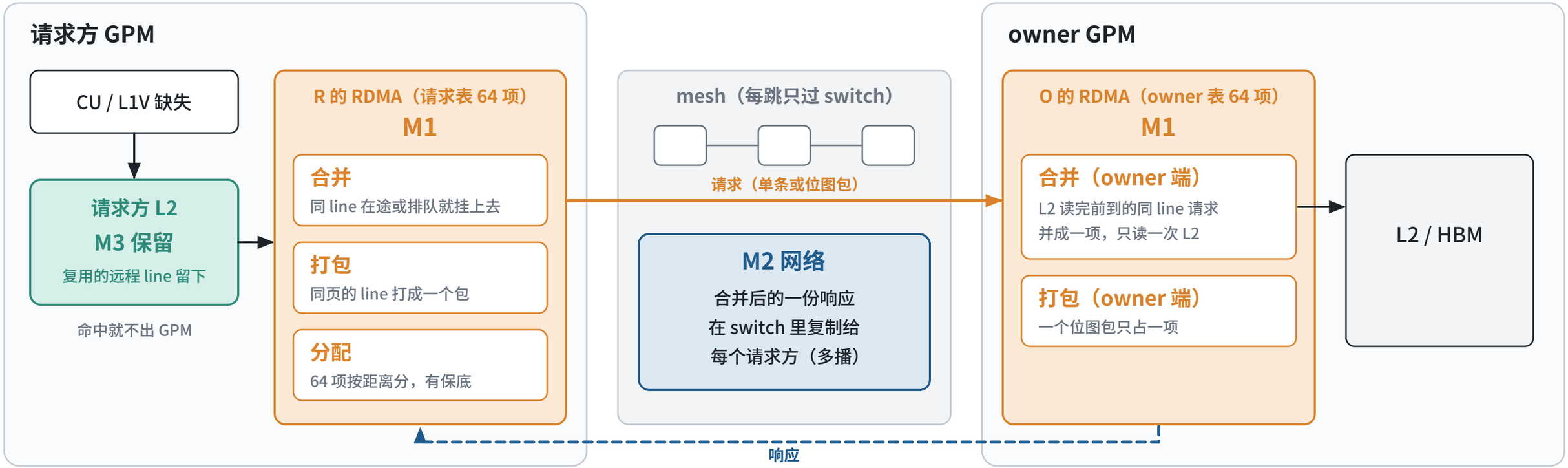
M1 RDMA 端：合并（同 line 两端各只占一项）、打包（同页一个包）、分配（64 项按距离分）

M2 网络：合并后的一份响应在 switch 里复制给每个请求方（网内多播）

M3 保留：远程数据看到复用才留在请求方 L2（非重点）

图 3 三个机制按硬件位置分：RDMA 端、网络、L2

都对着同一个瓶颈：两端 RDMA 各 64 项的表（链路利用率 < 3%）

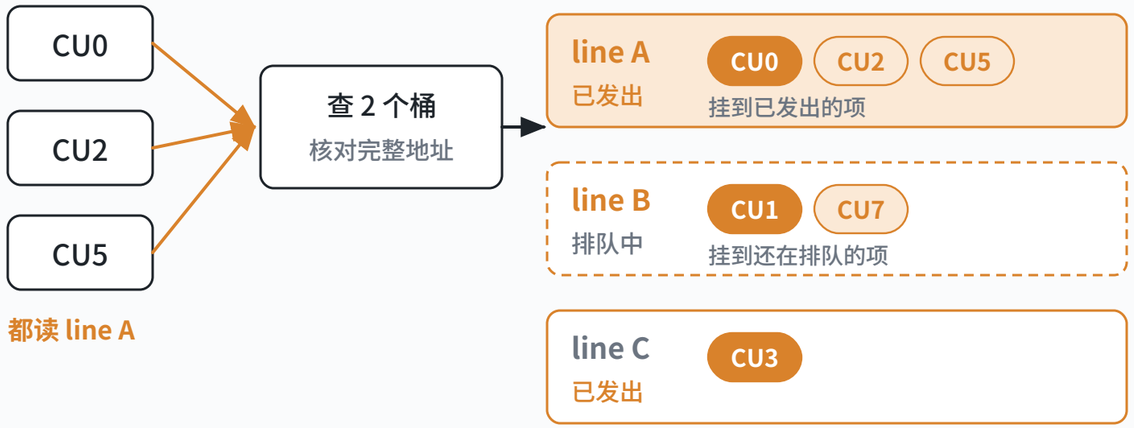


机制	硬件位置	做什么	对应的 wafer 特点	来源
M1 RDMA 端	两端各 64 项的表	合并 · 打包 · 分配	读者多：一条 line 最多 47 个（MCM 3 个） 距离差：往返 155–941 ns，6 倍（MCM 1.5 倍）	合并、分配：新（T18） 打包：HPCA 版
M2 网络	switch	网内多播 （建立在 M1 的 owner 合并上）	读者多、平均 4.3 跳	新（T18）
M3 保留	请求方 L2	看到复用才留下	远程数据会被同一 GPM 再读	HPCA 版，非重点

图 4 M1 合并：同一条 line 在两端各只占一项

请求方：在途表里已有这条 line 就挂上去，不再发；owner：L2 读完之前到的同 line 请求并成一项（T18 的实现）

请求方 RDMA：在途表（cuckoo 索引，128 槽）

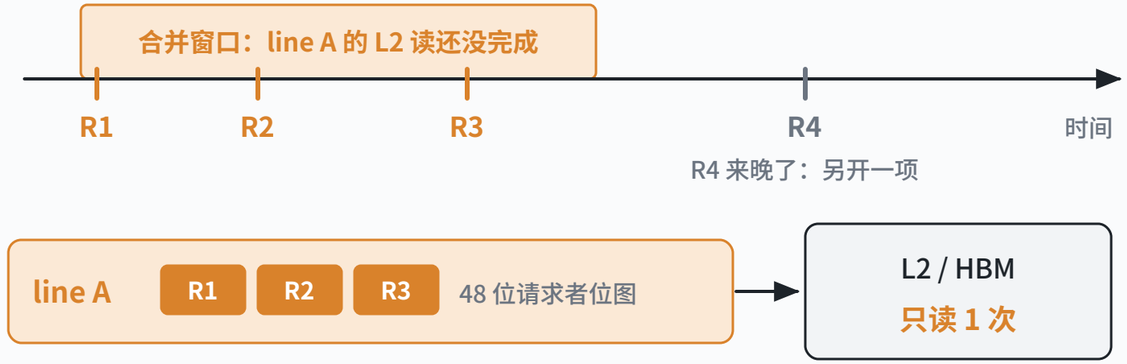


实心：发出去的那个；浅色：挂上去的等待者

网络上同时在途仍 ⩽ 64；另外 64 个槽给还在排队的请求，排队时也能合并

写会关闭这条 line 的项：写之后的读不会并到写之前的读上

owner RDMA：合并表（cuckoo 索引，128 槽）



3 个请求只占 owner 表 1 项、只读 1 次 L2

回给 R1–R3：M2 多播一份；不开 M2 就逐个单播

表满、同一请求者重复、不是整行读：照常单独处理

没有等待计时器，响应不会为了等更多请求而推迟

现有数据（FW 冒烟，64 项）

只开请求方合并：1.82×

只开 owner 合并：1.01×（几乎没用）

T18 @64（两端合并 + 分配 + M2 多播）：2.13×

旧 m2（去重 + 打包）：2.51×

为什么对 wafer 有用（P12）

同一 GPM 内同 line 已在途：FW 49%，R-GAT 34%

跨 GPM 同时在等同一 line：PR 79%，R-GAT 24%

一条 line 最多 47 个读者（MCM 最多 3 个）

PagedAttention 几乎没有（< 1%）

图 5 M1 打包：同页的 line 打成一个包，一个表项带多条 line

HPCA 版已有（旧 m2 的同页打包 + 预取捎带）；包在等表项时顺便收 line，所以表项越紧，包越满



规则

同一 owner、同一 4 KB 页；每包最多 8 条，最多 64 个包在收集

不设等待计时器：包在等表项或发射宽度时顺便收

预取：步长预测器 256 项，只在同 owner 同页里预测，

只捎在已有的包上

数据：表项越紧，包越满

64 项（HPCA 版 Complete）：平均每包 1.1–4.7 条

（allreduce 4.67、ivfflat 3.41、fft2d 3.10）

W = 1024（新 Complete）：只剩 1.0–2.1 条

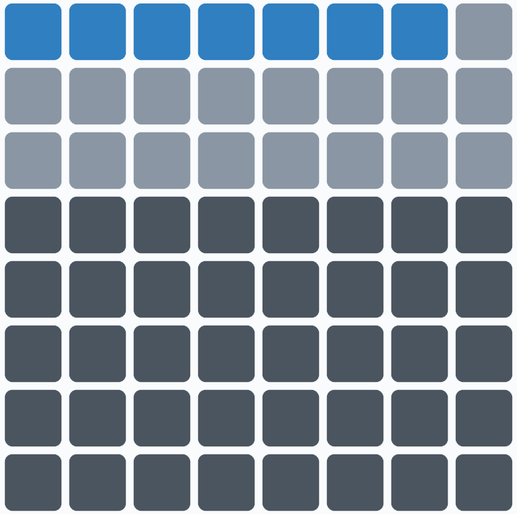
→ 回到 64 项后打包又重要；P15 会确认

图 6 M1 分配：64 项按距离分，每个目的地有保底，空闲的可以借

份额怎么定是参数：按往返时间分（T18 已有）或近处多留（新，要实现），用实验来选

按往返时间分（T18 的距离 credit，已有）

例子：角上的 GPM，47 个目的地分 64 项



近（≤ 3 跳）：7 项
9 个目的地

中（4–6 跳）：17 项
17 个目的地

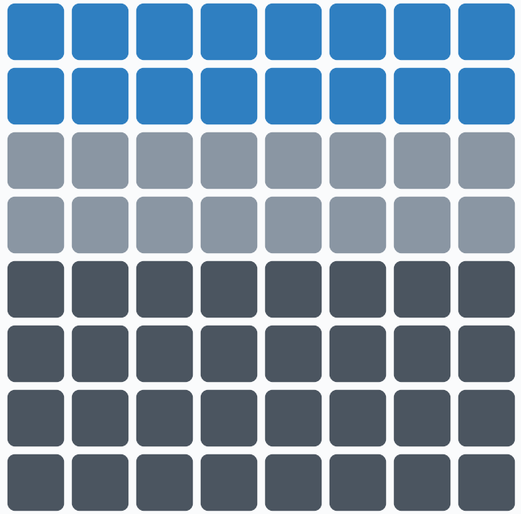
远（≥ 7 跳）：40 项
21 个目的地

1 跳的 2 个邻居保底是 0 项，
只能借别人空着的

目标：每个目的地拿到一样的带宽（Little 定律：远的每项占得久，所以多给）

近处多留（新，比例是示意）

近区单独留出来，远的不能借走



近区 16 项
只给 ≤ 3 跳的

其余 48 项
按往返时间分

目标：近的请求（155 ns 就能完成）不被远的挤在门口

两种份额共用的规则

有请求在等的目的地，
没用完的保底给它留着
空闲的可以借；借走的要等
响应回来才还（不抢回）

为什么是 wafer 才有的问题

往返 155–941 ns，差 6 倍
2×2 MCM 最多 2 跳，只差 1.5 倍
远的借一次要占约 800 ns
共享 64 项时远的占大头

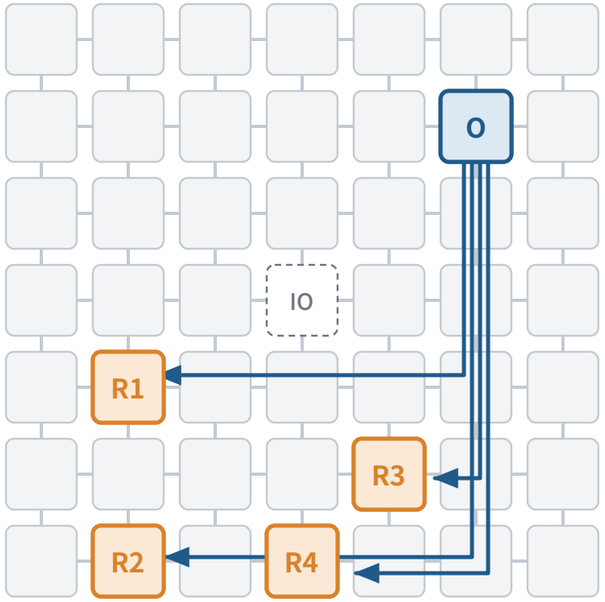
要先测

按距离分组的表占用（统计 ①）
Complete@64：credit 开 / 关
近处多留：在 t18_credits.go
加一种份额（Claude Code）

图 7 M2 网络：网内多播，合并后的一份响应在路由树上分叉

M1 在 owner 端合并后只发一份响应；switch 按 YX 路由复制给每个请求方；没合并的照常单播

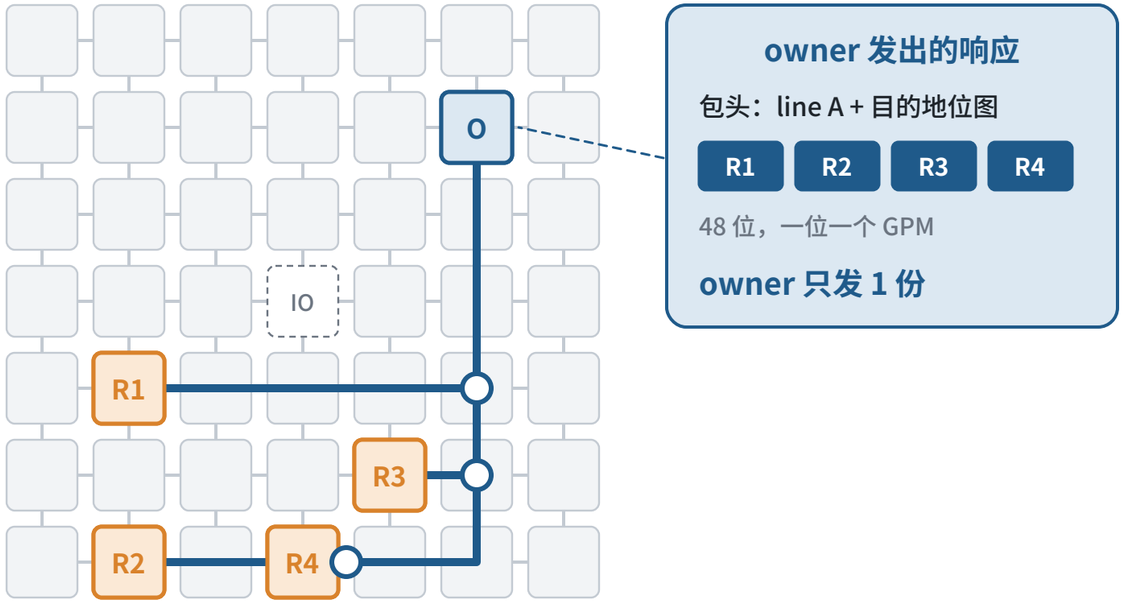
没有多播：4 份响应各走各的



owner 的 RDMA 发 4 份，共 28 段链路

同一段链路上最多叠 4 份同样的数据

多播：一份响应沿路由树分叉



一份响应在 switch 里复制（圆点处分叉），共 14 段链路

到了请求方 GPM 交给它的 RDMA，认领、唤醒等待者

switch 怎么做

1. 包头带目的地位图（来自 M1 的 owner 合并表）
2. 按 YX 路由把位图按输出端口拆开，各复制一份
3. 只有一个目的地，或表满没合并的：普通单播

为什么对 wafer 有用

一条 line 最多 47 个读者

读者之间最近只隔 1.5–2.6 跳

共享 line 上省 25–52% 链路

要说清楚的

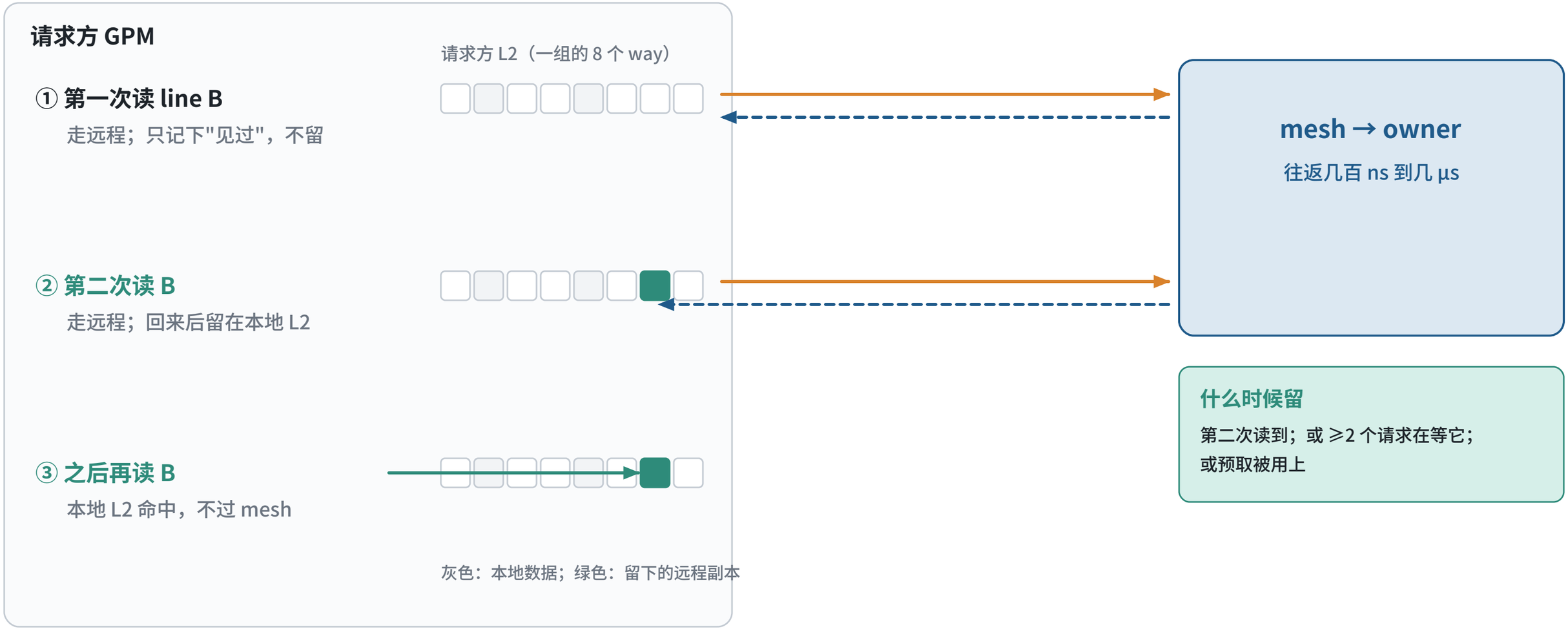
链路本来就闲（利用率 < 3%）

收益主要是 owner 少发几份响应

只有 M1 合并了的才能多播

图 8 M3 保留（HPCA 版 Remote Reuse，非重点）：看到复用才留在本地 L2

同一个 GPM 再次读到同一条远程 line 时，用本地 L2 里的副本，不再穿过晶圆

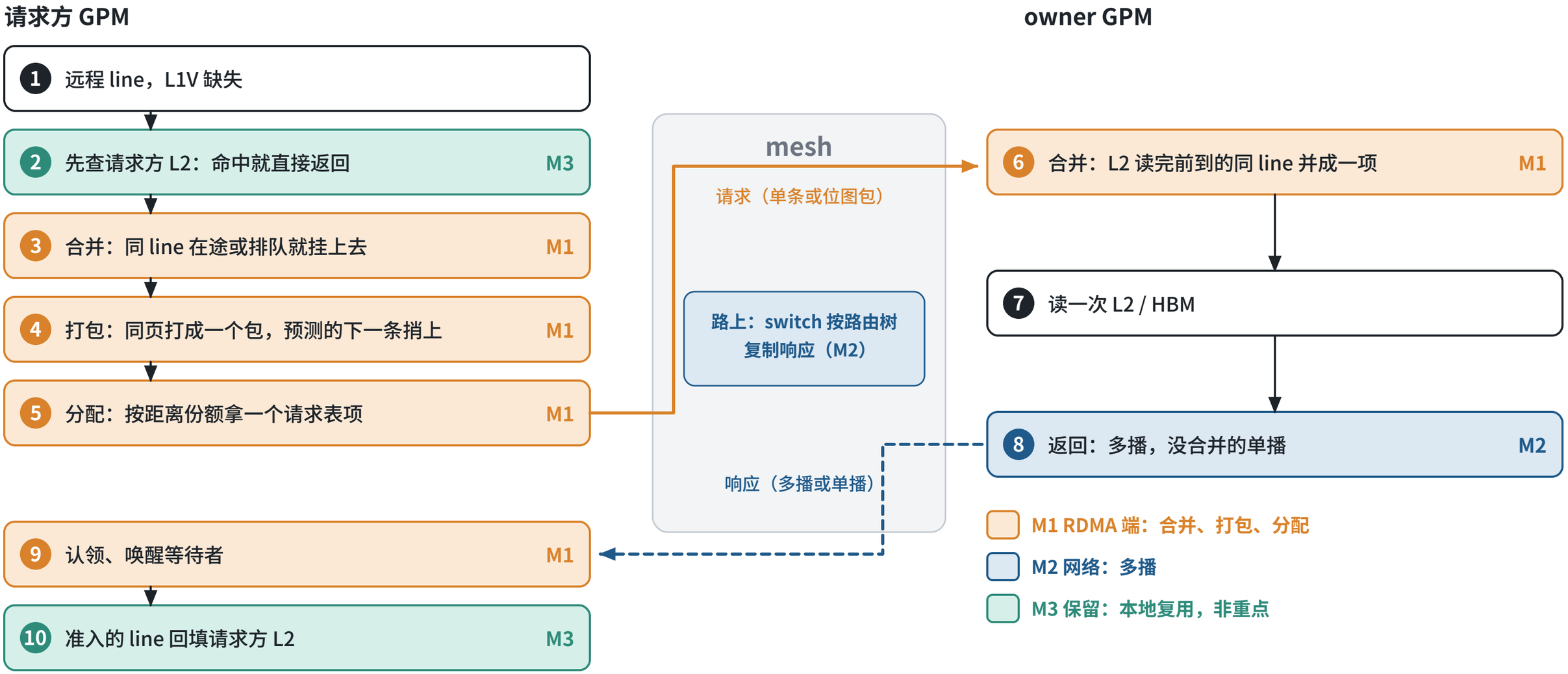


只占无效的 way 或替换别的远程副本，不挤掉本地数据；写照常发给 owner，本地副本一起更新。

还要补：kernel 边界的失效（论文写了，实现还没有）；模拟器的 L2 命中偏快（129 ns），M3 的收益可能偏高。

图 9 Complete = M1 + M2 + M3：一次远程读经过的每一步（RDMA 两端的表都是 64 项）

颜色表示这一步由哪个机制负责

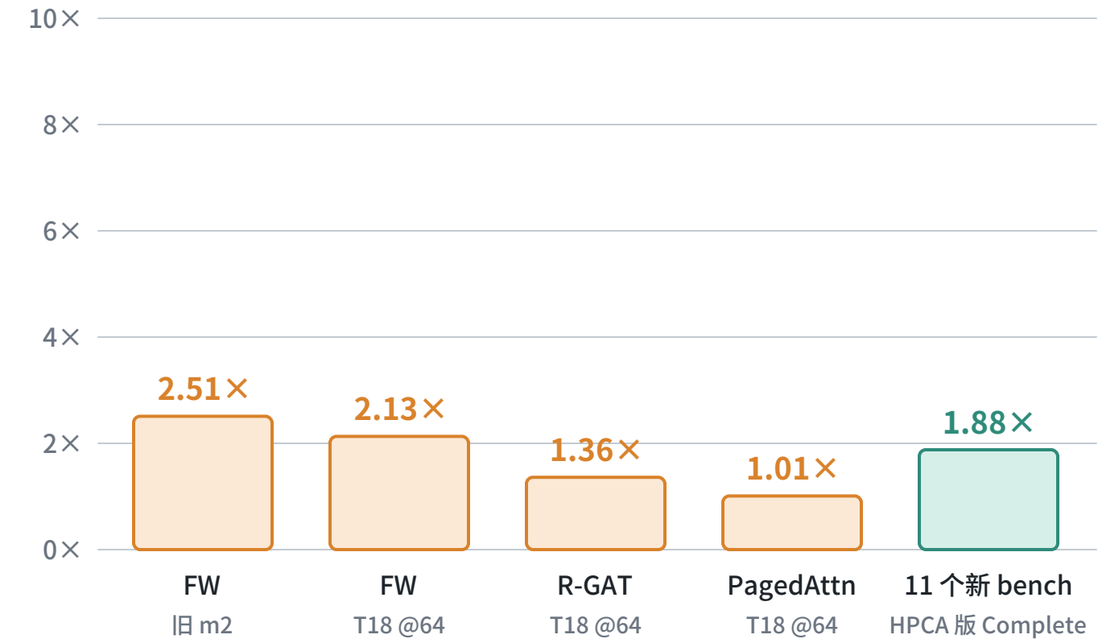


打包（第 4 步）和 M3（第 2、10 步）来自 HPCA 版；合并、分配、多播是新的（T18）；Local Pairing 不进论文

图 10 现有数据（表 64 项）和接下来要跑的

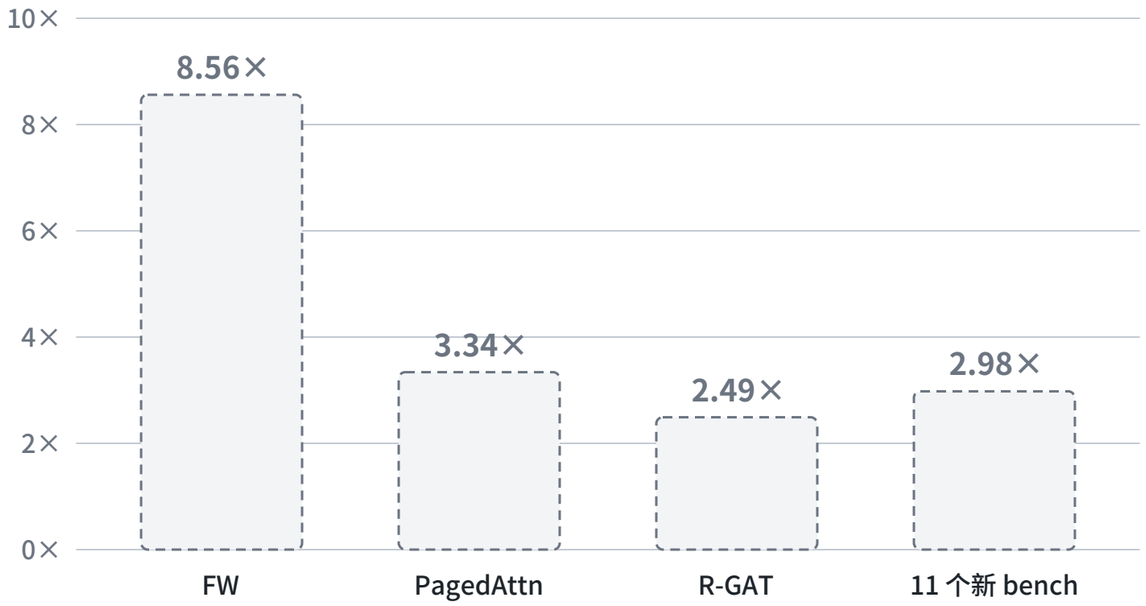
放大表项的结果只当"64 项这个限制值多少"的上限参考，不作为机制

(a) 表 64 项下的现有结果（冒烟）



旧 m2 = 去重 + 打包（现在属于 M1）；T18 @64 = M1 合并 + 分配 + M2 多播，不含打包

(b) 上限参考：表放大到 1024 项（不作为机制）



差距说明 64 项的限制很贵；三个机制要在不改 64 的前提下拿回一部分

接下来（表都保持 64 项）：

① P15（已写好）：Complete@64 vs baseline，15 个新 benchmark、30 个运行，和 W = 1024 的 2.82× 对比

② 消融：baseline、M1、M1 + M2、M1 + M3、Complete；M1 内部去掉合并 / 打包 / 分配各一次

③ 统计：按距离分组的表占用（分配）、owner 合并数和表满时间（合并）、多播目的地数和退回单播比例（M2）